

目 录

第 1 章 GPU 硬件与 CUDA 程序开发工具	1
1.1 GPU 硬件简介	1
1.2 CUDA 程序开发工具	4
1.3 CUDA 开发环境搭建示例	6
1.4 用 nvidia-smi 检查与设置设备	7
1.5 其他学习资料	8
第 2 章 CUDA 中的线程组织	10
2.1 C++ 语言中的 Hello World 程序	10
2.2 CUDA 中的 Hello World 程序	11
2.2.1 只有主机函数的 CUDA 程序	11
2.2.2 使用核函数的 CUDA 程序	12
2.3 CUDA 中的线程组织	14
2.3.1 使用多个线程的核函数	14
2.3.2 使用线程索引	15
2.3.3 推广至多维网格	17
2.3.4 网格与线程块大小的限制	21
2.4 CUDA 中的头文件	21
2.5 用 nvcc 编译 CUDA 程序	22
第 3 章 简单 CUDA 程序的基本框架	25
3.1 例子: 数组相加	25
3.2 CUDA 程序的基本框架	27
3.2.1 隐形的设备初始化	29
3.2.2 设备内存的分配与释放	29
3.2.3 主机与设备之间数据的传递	31
3.2.4 核函数中数据与线程的对应	32
3.2.5 核函数的要求	33
3.2.6 核函数中 if 语句的必要性	34
3.3 自定义设备函数	35

3.3.1	函数执行空间标识符	35
3.3.2	例子: 为数组相加的核函数定义一个设备函数	36
第 4 章	CUDA 程序的错误检测	38
4.1	一个检测 CUDA 运行时错误的宏函数	38
4.1.1	检查运行时 API 函数	40
4.1.2	检查核函数	42
4.2	用 CUDA-MEMCHECK 检查内存错误	44
第 5 章	获得 GPU 加速的关键	46
5.1	用 CUDA 事件计时	46
5.1.1	为 C++ 程序计时	47
5.1.2	为 CUDA 程序计时	48
5.2	几个影响 GPU 加速的关键因素	50
5.2.1	数据传输的比例	50
5.2.2	算术强度	51
5.2.3	并行规模	54
5.2.4	总结	55
5.3	CUDA 中的数学函数库	55
第 6 章	CUDA 的内存组织	57
6.1	CUDA 的内存组织简介	57
6.2	CUDA 中不同类型的内存	58
6.2.1	全局内存	58
6.2.2	常量内存	61
6.2.3	纹理内存和表面内存	62
6.2.4	寄存器	62
6.2.5	局部内存	63
6.2.6	共享内存	63
6.2.7	L1 和 L2 缓存	64
6.3	SM 及其占有率	65
6.3.1	SM 的构成	65
6.3.2	SM 的占有率	65
6.4	用 CUDA 运行时 API 函数查询设备	67
第 7 章	全局内存的合理使用	70
7.1	全局内存的合并与非合并访问	70
7.2	例子: 矩阵转置	73
7.2.1	矩阵复制	73

7.2.2	使用全局内存进行矩阵转置	75
第 8 章	共享内存的合理使用	78
8.1	例子: 数组归约计算	78
8.1.1	仅使用全局内存	79
8.1.2	使用共享内存	82
8.1.3	使用动态共享内存	84
8.2	使用共享内存进行矩阵转置	85
8.3	避免共享内存的 bank 冲突	86
第 9 章	原子函数的合理使用	90
9.1	完全在 GPU 中进行归约	90
9.2	原子函数	93
9.3	例子: 邻居列表的建立	95
9.3.1	C++ 版本的开发	96
9.3.2	利用原子操作的 CUDA 版本	98
9.3.3	不用原子操作的 CUDA 版本	101
第 10 章	线程束基本函数与协作组	104
10.1	单指令-多线程执行模式	104
10.2	线程束内的线程同步函数	106
10.3	更多线程束内的基本函数	109
10.3.1	介绍	109
10.3.2	利用线程束洗牌函数进行归约计算	114
10.4	协作组	116
10.4.1	线程块级别的协作组	116
10.4.2	利用协作组进行归约计算	118
10.5	数组归约程序的进一步优化	119
10.5.1	提高线程利用率	119
10.5.2	避免反复分配与释放设备内存	122
第 11 章	CUDA 流	124
11.1	CUDA 流概述	124
11.2	在默认流中重叠主机和设备计算	125
11.3	用非默认 CUDA 流重叠多个核函数的执行	128
11.3.1	核函数执行配置中的流参数	128
11.3.2	重叠多个核函数的例子	129
11.4	用非默认 CUDA 流重叠核函数的执行与数据传递	131
11.4.1	不可分页主机内存与异步的数据传输函数	131

11.4.2	重叠核函数执行与数据传输的例子	133
第 12 章	使用统一内存编程	136
12.1	统一内存简介	136
12.1.1	统一内存的基本概念	136
12.1.2	使用统一内存对硬件的要求	137
12.1.3	统一内存编程的优势	137
12.2	统一内存的基本使用方法	137
12.2.1	动态统一内存	138
12.2.2	静态统一内存	139
12.3	使用统一内存申请超量的内存	140
12.3.1	第一个测试	140
12.3.2	第二个测试	142
12.3.3	第三个测试	143
12.4	优化使用统一内存的程序	144
第 13 章	分子动力学模拟的 CUDA 程序开发	147
13.1	分子动力学模拟的基本算法和 C++ 实现	147
13.1.1	程序的整体结构	147
13.1.2	分子动力学模拟的基本流程	148
13.1.3	初始条件	149
13.1.4	边界条件	150
13.1.5	相互作用	152
13.1.6	运动方程的数值积分	156
13.1.7	程序中使用的单位制	157
13.1.8	程序的编译与运行	158
13.1.9	能量守恒的测试	159
13.1.10	C++ 版本程序运行速度的测试	160
13.2	CUDA 版本的分子动力学模拟程序开发	161
13.2.1	仅加速求力和能量的部分	161
13.2.2	加速全部计算	165
第 14 章	CUDA 标准库的使用	167
14.1	CUDA 标准库简介	167
14.2	Thrust 库	168
14.2.1	简介	168
14.2.2	数据结构	168
14.2.3	算法	168

14.2.4	例子: 前缀和	169
14.3	cuBLAS 库	171
14.3.1	简介	171
14.3.2	例子: 矩阵乘法	172
14.4	cuSolver 库	176
14.4.1	简介	176
14.4.2	例子: 矩阵本征值	177
14.5	cuRAND 库	181
14.5.1	简介	181
14.5.2	例子	182

第 1 章

GPU硬件与CUDA程序开发工具

1.1 GPU 硬件简介

GPU 是英文 graphics processing unit 的首字母缩写, 意为图形处理器。GPU 也被称为显卡 (graphics card)。与它对应的一个概念是 CPU, 即 central processing unit (中央处理器) 的首字母缩写。

从十多年前起, GPU 的浮点数运算峰值就比同时期的 CPU 高一个量级; GPU 的内存带宽峰值也比同时期的 CPU 高一个量级。CPU 和 GPU 的显著区别如下: 一块典型的 CPU 拥有少数几个快速的计算核心, 而一块典型的 GPU 拥有几百到几千个不那么快速的计算核心。CPU 中有更多的晶体管用于数据缓存和流程控制, 但 GPU 中有更多的晶体管用于算术逻辑单元。所以, GPU 是靠众多的计算核心来获得相对较高的计算性能的。图 1.1 形象地说明了 (非集成) GPU 和 CPU 在硬件架构上的显著区别。

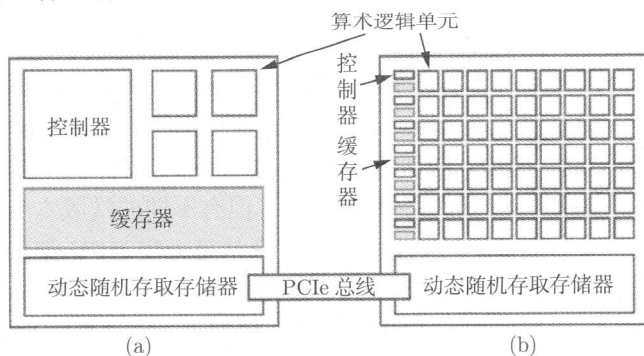


图 1.1 CPU 和非集成 GPU 的硬件架构示意图

(a) CPU; (b) GPU

CPU 和 GPU 中都有 DRAM (dynamic random access memory, 动态随机存取存储器), 它们一般由 PCIe 总线 (peripheral component interconnect express bus) 连接。

GPU 计算不是指单独的 GPU 计算, 而是指 CPU + GPU 的异构 (heterogeneous) 计算。一块单独的 GPU 是无法独立地完成所有计算任务的, 它必须在 CPU

的调度下才能完成特定任务。在由 CPU 和 GPU 构成的异构计算平台中,通常将起控制作用的 CPU 称为主机(host),将起加速作用的 GPU 称为设备(device)。主机和(非集成)设备都有自己的 DRAM,它们之间一般由 PCIe 总线连接,如图 1.1 所示。

本书中说的 GPU 都是指英伟达(Nvidia)公司推出的 GPU,因为 CUDA 编程目前只支持该公司的 GPU。以下几个系列的 GPU 都支持 CUDA 编程。

(1) Tesla 系列:其中的内存为纠错内存(error-correcting code memory, ECC 内存),稳定性好,主要用于高性能、高强度的科学计算。

(2) Quadro 系列:支持高速 OpenGL(open graphics library)渲染,主要用于专业绘图设计。

(3) GeForce 系列:主要用于游戏与娱乐,但也常用于科学计算。GeForce 系列的 GPU 没有纠错内存,用于科学计算时具有一定的风险。然而,GeForce 系列的 GPU 价格相对低廉、性价比高,用于学习 CUDA 编程是没有任何问题的。即使是便携式计算机中 GeForce 系列的 GPU 也可以用来学习 CUDA 编程。

(4) Jetson 系列:嵌入式设备中的 GPU。作者对此无使用经验,本书也不专门讨论。

每一款 GPU 都有一个用以表示其“计算能力”(compute capability)的版本号。该版本号可以写为形如 $X.Y$ 的形式。其中, X 表示主版本号, Y 表示次版本号。版本号决定了 GPU 硬件所支持的功能,可为应用程序在运行时判断硬件特征提供依据。初学者往往误以为 GPU 的计算能力越高,性能就越高,但后面我们会看到,计算能力和性能没有简单的正比关系。

版本号越大的 GPU 架构(architecture)越新。主版本号与 GPU 的核心架构相关联。英伟达公司选择用著名科学家(到目前为止,大部分是物理学家)的姓氏作为 GPU 核心架构的代号,见表 1.1。在主版本号相同时,具有较大次版本号的 GPU 的架构稍有更新。例如,同属于开普勒(Kepler)架构的 Tesla K40 和 Tesla K80 这两款 GPU 有相同的主版本号($X=3$),但有不同的次版本号,它们的计算能力分别是 3.5 和 3.7。注意:特斯拉(Tesla)既是第一代 GPU 架构的代号,也是科学计算系列 GPU 的统称,其具体含义要根据上下文确定。另外,计算能力为 7.5 的架构虽然和伏特(Volta)架构具有同样的主版本号($X=7$),但它一般被看作一个新的主要架构,代号为图灵(Turing)。表 1.2 列出了不同架构的各种 GPU 的名称。

特斯拉架构和费米(Fermi)架构的 GPU 已不再受到最近几个 CUDA 版本的支持。本书将忽略任何特定于这两个架构的硬件功能。可以预见,开普勒架构的 GPU 也将很快(如一两年后)不受最新版 CUDA 的支持。为简洁起见,本书有时也将忽略某些开普勒架构的特征。为简单起见,我们在表 1.2 中忽略了一类被称

为 Titan 的 GPU。读者可以在如下网站查询任何一款支持 CUDA 的 GPU 的信息：<http://developer.nvidia.com/cuda-gpus>。

表 1.1 各个 GPU 主计算能力的架构代号与发布年份

主计算能力	架构代号	发布年份
$X = 1$	特斯拉 (Tesla)	2006
$X = 2$	费米 (Fermi)	2010
$X = 3$	开普勒 (Kepler)	2012
$X = 5$	麦克斯韦 (Maxwell)	2014
$X = 6$	帕斯卡 (Pascal)	2016
$X = 7$	伏特 (Volta)	2017
$X.Y = 7.5$	图灵 (Turing)	2018

表 1.2 当前常用的各种 GPU 的名称

架构	Tesla 系列	Quadro 系列	GeForce 系列	Jetson 系列
开普勒	Tesla K 系列	Quadro K 系列	GeForce 600/700 系列	Tegra K1
麦克斯韦	Tesla M 系列	Quadro M 系列	GeForce 900 系列	Tegra X1
帕斯卡	Tesla P 系列	Quadro P 系列	GeForce 1000 系列	Tegra X2
伏特	Tesla V 系列	无	无	AGX Xavier
图灵	Tesla T 系列	Quadro RTX 系列	GeForce 2000 系列	AGX Xavier

注：特斯拉架构和费米架构的 GPU 已经不再受到最新 CUDA 的支持，故没有列出。

计算能力并不等价于计算性能。例如，GeForce RTX 2000 系列的计算能力高于 Tesla V100，但后者在很多方面性能更高（售价也高得多）。

表征计算性能的一个重要参数是浮点数运算峰值，即每秒最多能执行的浮点数运算次数（floating-point operations per second, FLOPS）。GPU 的浮点数运算峰值在 10^{12} FLOPS，即 teraFLOPS（简称为 TFLOPS）的量级。浮点数运算峰值有单精度和双精度之分。对 Tesla 系列的 GPU 来说，双精度浮点数运算峰值一般是单精度浮点数运算峰值的 1/2 左右（对计算能力为 3.5 和 3.7 的 GPU 来说，是 1/3 左右）。对 GeForce 系列的 GPU 来说，双精度浮点数运算峰值一般是单精度浮点数运算峰值的 1/32 左右。另一个影响计算性能的参数是 GPU 中的内存带宽（memory bandwidth）。GPU 中的内存常称为显存。显存容量也是制约应用程序性能的一个因素。如果一个应用程序需要的显存数量超过了一个 GPU 的显存容量，则在不使用统一内存（见第 12 章）的情况下程序就无法正确运行。表 1.3 列出了作者目前能够使用的几款 GPU 的主要性能指标。在浮点数运算峰值一栏中，括号前和括号中的数字分别对应双精度和单精度的情形。

表 1.3 若干 GPU 的主要性能指标

GPU 型号	计算能力	显存容量/GB	显存带宽/(GB/s)	浮点数运算 峰值/TFLOPS
Tesla K40	3.5	12	288	1.4 (4.3)
Tesla P100	6.0	16	732	4.7 (9.3)
Tesla V100	7.0	32	900	7 (14)
GeForce RTX 2070	7.5	8	448	0.2 (6.5)
GeForce RTX 2080ti	7.5	11	616	0.4 (13)

1.2 CUDA 程序开发工具

以下几种软件开发工具都可以用来进行 GPU 编程。

(1) CUDA。这是本书的主题。

(2) OpenCL。这是一个更为通用的为各种异构平台编写并行程序的框架，也是 AMD (Advanced Micro Devices) 公司的 GPU 的主要程序开发工具。本书不涉及 OpenCL 编程，对此感兴趣的读者可参考《OpenCL 异构并行计算：原理、机制与优化实践》(刘文志，陈轶，吴长江，北京：机械工业出版社)。

(3) OpenACC。这是一个由多个公司共同开发的异构并行编程标准。本书也不涉及 OpenACC 编程，对此感兴趣的读者可参考《OpenACC 并行编程实战》(何沧平，北京：机械工业出版社)。

CUDA 编程语言最初主要是基于 C 语言的，但目前越来越多地支持 C++ 语言。另外，还有基于 Fortran 的 CUDA Fortran 版本及由其他编程语言包装的 CUDA 版本，但本书只涉及基于 C++ 的 CUDA 编程。我们称基于 C++ 的 CUDA 编程语言为 CUDA C++。对 Fortran 版本感兴趣的读者可以参考网站 <https://www.pgroup.com/> 中的资源。用户可以免费下载支持 CUDA Fortran 编程的 PGI 开发工具套装的社区版本 (community edition)。对应的还有收费的专业版本 (professional edition)。PGI 是高性能计算编译器公司 Portland Group, Inc. 的简称，已被英伟达公司收购。

CUDA 提供了两层 API (application programming interface, 应用程序编程接口) 供程序员使用，即 CUDA 驱动 (driver) API 和 CUDA 运行时 (runtime) API。其中，CUDA 驱动 API 是更加底层的 API，它为程序员提供了更为灵活的编程接口；CUDA 运行时 API 是在 CUDA 驱动 API 的基础上构建的一个更为高级的 API，更容易使用。这两种 API 在性能上几乎没有差别。从程序的可读性来看，使用 CUDA 运行时 API 是更好的选择。在其他编程语言中使用 CUDA 时，驱动 API 很多时候是必需的。因为作者没有使用驱动 API 的经验，故本书只涉及 CUDA 运

行时 API。

图 1.2 展示了 CUDA 开发环境的主要组件。开发的应用程序是以主机 (CPU) 为出发点的。应用程序可以调用 CUDA 运行时 API、CUDA 驱动 API 及一些已有的 CUDA 库。所有这些调用都将利用设备 (GPU) 的硬件资源。对 CUDA 运行时 API 的介绍是本书大部分章节的重点内容; 第 14 章将介绍若干常用的 CUDA 库。

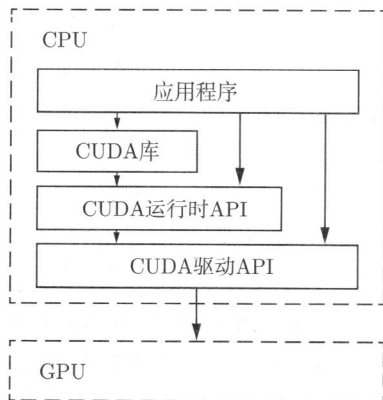


图 1.2 CUDA 编程开发环境概览

CUDA 版本也由形如 $X.Y$ 的两个数字表示, 但它并不等同于 GPU 的计算能力。可以这样理解: CUDA 版本是 GPU 软件开发平台的版本, 而计算能力对应着 GPU 硬件架构的版本。

最早的 CUDA 1.0 于 2007 年发布。当前 (笔者交稿之日) 最新的版本是 CUDA 10.2。CUDA 版本与 GPU 的最高计算能力都在逐年上升。虽然它们之间没有严格的对应关系, 但一个具有较高计算能力的 GPU 通常需要一个较高的 CUDA 版本才能支持。最近的几个 CUDA 版本对 GPU 计算能力的支持情况见表 1.4。一般来说, 建议安装一个支持所用 GPU 的较新的 CUDA 工具箱。本书中的所有示例程序都可以在 CUDA 9.0~10.2 中进行测试。目前最新版本的 CUDA 10.2 有两个值得注意的地方。第一, 它是最后一个支持 MacOS 的 CUDA 版本; 第二, 它将 CUDA C 改名为 CUDA C++, 用以强调 CUDA C++ 是基于 C++ 的扩展。

表 1.4 最近的几个 CUDA 版本对 GPU 计算能力的支持情况

CUDA 版本	所支持 GPU 的计算能力	架构
10.0~10.2	3.0~7.5	从开普勒到图灵
9.0~9.2	3.0~7.2	从开普勒到伏特
8.0	2.0~6.2	从费米到帕斯卡
7.0~7.5	2.0~5.3	从费米到麦克斯韦

1.3 CUDA 开发环境搭建示例

下面叙述作者最近在装有 GeForce RTX 2070 的便携式计算机（以下简称计算机）中搭建 CUDA 开发环境的大致过程。因为作者的计算机中预装了 Windows 10 操作系统，所以我们以 Windows 10 操作系统为例进行讲解。因为 Linux 发行版有多种，故本书不列出在 Linux 中安装 CUDA 开发环境的步骤。读者可参阅英伟达公司的官方文档：<https://docs.nvidia.com/cuda/cuda-installation-guide-linux>。

我们说过，GPU 计算实际上是 CPU+GPU（主机 + 设备）的异构计算。在 CUDA C++ 程序中，既有运行于主机的代码，又有运行于设备的代码。其中，运行于主机的代码需要由主机的 C++ 编译器编译和链接。所以，除安装 CUDA 工具箱外，还需要安装一个主机的 C++ 编译器。在 Windows 中，最常用的 C++ 编译器是 Microsoft Visual C++ (MSVC)，它目前集成在 Visual Studio 中，所以我们首先安装 Visual Studio。作者安装了最高版本的 Visual Studio 2019 16.x。因为这是个人使用的，故选择了免费的 Community 版本。下载地址为 <https://visualstudio.microsoft.com/free-developer-offers/>。对于 CUDA C++ 程序开发来说，只需要选择安装 Desktop development with C++ 即可。当然，读者也可以选择安装更多的组件。

关于 CUDA，作者选择安装 2019 年 8 月发布的 CUDA Toolkit 10.1 update2。首先，进入网址 <https://developer.nvidia.com/cuda-10.1-download-archive-update2>。然后，根据提示，做如下选择：Operating System 项选择 Windows；Architecture 项选择 x86_64；Version 项选择操作系统版本，我们这里是 10；Installer Type 项可以选择 exe (network) 或者 exe (local)，分别代表一边下载一边安装和下载完毕后安装。最后，运行安装程序，根据提示一步一步安装即可。该版本的 CUDA 工具箱包含一个对应版本的 Nvidia driver，故不需要再单独安装 Nvidia driver。

安装好 Visual Studio 和 CUDA 后，进入如下目录（读者如果找不到 C 盘下的 ProgramData 目录，可能是因为没有选择显示一些隐藏的文件）：

```
C:\ProgramData\NVIDIA Corporation\CUDA Samples\v10.1\1-Utilities  
  \deviceQuery。
```

用 Visual Studio 2019 打开文件 deviceQuery_vs2019.sln，编译（构建）、运行即可。若输出内容的最后部分为 Result=PASS，则说明已经搭建好 Windows 中的 CUDA 开发环境。若有疑问，请参阅英伟达公司的官方文档：<https://docs.nvidia.com/cuda/cuda-installation-guide-microsoft-windows>。

在上面的测试中，我们是直接用 Visual Studio 打开一个已有的解决方案（solution），然后直接构建并运行。本书不介绍 Visual Studio 的使用，而是选择用命令行解释器编译与运行程序。这里的命令行解释器指的是 Linux 中的 termi-

nal 或者 Windows 中的 command prompt 程序。在 Windows 中使用 MSVC 作为 C++ 程序的编译器时, 需要单独设置相应环境变量, 或者从 Windows 的“开始”(start) 菜单中找到 Visual Studio 2019 文件夹, 然后单击其中的“x64 Native Tools Command Prompt for VS 2019”, 从而打开一个加载了 MSVC 环境变量的命令行解释器。在本书的某些章节, 需要有管理员的权限来使用 nvprof 性能分析器。此时, 可以右击“x64 Native Tools Command Prompt for VS 2019”, 在弹出的快捷菜单中选择“更多”→“以管理员身份运行”。

用命令行解释器编译与运行 CUDA 程序的方式在 Windows 和 Linux 系统几乎没有区别, 但为了简洁起见, 本书后面主要以 Linux 开发环境为例进行讲解。虽然如此, Windows 和 Linux 中的 CUDA 编程功能还是稍有差别。我们将在后续章节中适当的地方指出这些差别。

1.4 用 nvidia-smi 检查与设置设备

可以通过 nvidia-smi (Nvidia's system management interface) 程序检查与设置设备。它包含在 CUDA 开发工具套装内。该程序最基本的用法就是在命令行解释器中使用不带任何参数的命令 nvidia-smi。在作者的计算机中使用该命令, 得到如下文本形式的输出:

```
+-----+
| NVIDIA-SMI 426.00      Driver Version: 426.00      CUDA Version: 10.1      |
+-----+
| GPU  Name           TCC/WDDM | Bus-Id       Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf   Pwr:Usage/Cap|      Memory-Usage | GPU-Util  Compute M. |
+-----+-----+
|   0  GeForce RTX 207... WDDM | 00000000:01:00:0 Off |              N/A     |
| N/A   38C    P8     12W /  N/A |   161MiB /   8192MiB |      0%      Default  |
+-----+-----+

+-----+
| Processes:                                     GPU Memory |
|  GPU       PID    Type    Process name                     Usage      |
+-----+-----+
| No running processes found                                     |
+-----+
```

从中可以看出一些比较有用的信息:

- (1) 第一行可以看到 Nvidia driver 的版本及 CUDA 工具箱的版本。
- (2) 作者所用计算机中有一型号为 GeForce RTX 2070 的 GPU。该 GPU 的设备号是 0。该计算机仅有一个 GPU。如果有多个 GPU, 会将各个 GPU 从 0 开始

编号。如果读者的系统中有多 GPU，而且只需要使用某个特定的 GPU（如两个之中更强大的那个），则可以通过设置环境变量 `CUDA_VISIBLE_DEVICES` 的值在运行 CUDA 程序之前选定一个 GPU。假如读者的系统中有编号为 0 和 1 的两个 GPU，而读者想在 1 号 GPU 运行 CUDA 程序，可以用如下命令设置环境变量：

```
$ export CUDA_VISIBLE_DEVICES=1
```

这样设置的环境变量在当前 shell session 及其子进程中有效。

(3) 该 GPU 处于 WDDM (Windows display driver model) 模式。另一个可能的模式是 TCC (Tesla compute cluster)，但它仅在 Tesla、Quadro 和 Titan 系列的 GPU 中可选。可用如下方式选择（在 Windows 中需要用管理员身份打开 Command Prompt 并去掉命令中的 `sudo`）：

```
$ sudo nvidia-smi -g GPU_ID -dm 0 # 设置为 WDDM 模式
```

```
$ sudo nvidia-smi -g GPU_ID -dm 1 # 设置为 TCC 模式
```

这里，`GPU_ID` 是 GPU 的编号。

(4) 该 GPU 当前的温度为 38°C。GPU 在满负荷运行时，温度会高一些。

(5) 这是 GeForce 系列的 GPU，没有 ECC 内存，故 Uncorr. ECC 为 N/A，代表不适用 (not applicable) 或者不存在 (not available)。

(6) Compute M. 指计算模式 (compute mode)。该 GPU 的计算模式是 Default。在默认模式中，同一个 GPU 中允许存在多个计算进程，但每个计算进程对应程序的运行速度一般来说会降低。还有一种模式为 E. Process，指的是独占进程模式 (exclusive process mode)，但不适用于处于 WDDM 模式的 GPU。在独占进程模式下，只能运行一个计算进程独占该 GPU。可以用如下命令设置计算模式（在 Windows 中需要用管理员身份打开 Command Prompt 并去掉命令中的 `sudo`）：

```
$ sudo nvidia-smi -i GPU_ID -c 0 # 默认模式
```

```
$ sudo nvidia-smi -i GPU_ID -c 1 # 独占进程模式
```

这里，`-i GPU_ID` 的意思是希望该设置仅仅作用于编号为 `GPU_ID` 的 GPU；如果去掉该选项，该设置将会作用于系统中所有的 GPU。

关于 `nvidia-smi` 程序的更多介绍，请参考如下官方文档：<https://developer.nvidia.com/nvidia-system-management-interface>。

1.5 其他学习资料

本书将循序渐进地带领读者学习 CUDA C++ 编程的基础知识。虽然作者力求本书知识全面、内容系统，但读者在阅读本书的过程中同时参考一些其他的学习资料也是有好处的。

任何关于 CUDA 编程的书籍都不可能替代官方提供的手册等资料。以下是几个重要的官方文档,请读者在有一定的基础之后务必查阅。限于作者水平,本书难免存在不当之处。当读者觉得本书中的个别论断与官方资料有冲突时,当以官方资料为标准(官方手册的网址为<https://docs.nvidia.com/cuda>)。在这个网站,包括但不限于以下几个方面文档:

(1) 安装指南(installation guides)。读者遇到与 CUDA 安装有关的问题时,应该仔细阅读此处的文档。

(2) 编程指南(programming guides)。该部分有很多重要的文档:

1) 最重要的文档是《CUDA C++ Programming Guide》,网址为<https://docs.nvidia.com/cuda/cuda-c-programming-guide>。

2) 另一个值得一看的文档是《CUDA C++ Best Practices Guide》,网址为<https://docs.nvidia.com/cuda/cuda-c-best-practices-guide>。

3) 针对最近的几个 GPU 架构进行优化的指南,网址为

① <https://docs.nvidia.com/cuda/kepler-tuning-guide>。

② <https://docs.nvidia.com/cuda/maxwell-tuning-guide>。

③ <https://docs.nvidia.com/cuda/pascal-tuning-guide>。

④ <https://docs.nvidia.com/cuda/volta-tuning-guide>。

⑤ <https://docs.nvidia.com/cuda/turing-tuning-guide>。

这几个简短的文档可以帮助有经验的用户迅速了解一个新的架构。

(3) CUDA API 手册(CUDA API references)。这里有

1) CUDA 运行时 API 的手册: <https://docs.nvidia.com/cuda/cuda-runtime-api>。

2) CUDA 驱动 API 的手册: <https://docs.nvidia.com/cuda/cuda-driver-api>。

3) CUDA 数学函数库 API 的手册: <https://docs.nvidia.com/cuda/cuda-math-api>。

4) 其他若干 CUDA 库的手册。

为明确起见,在撰写本书时,作者参考的是与 CUDA 10.2 对应的官方手册。

第 6 章

CUDA的内存组织

前一章讨论了几个获得 GPU 加速的必要但不充分条件。在满足那些条件之后，要获得尽可能高的性能，还有很多需要注意的方面，其中最重要的是合理地使用各种设备内存。本章从整体上介绍 CUDA 中的内存组织，为后续章节的讨论打好理论基础。

6.1 CUDA 的内存组织简介

现代计算机中的内存往往存在一种组织结构（hierarchy）。在这种结构中，含有多多种类型的内存，每种内存分别具有不同的容量和延迟（latency，可以理解为处理器等待内存数据的时间）。一般来说，延迟低（速度高）的内存容量小，延迟高（速度低）的内存容量大。当前被处理的数据一般存放于低延迟、低容量的内存中；当前没有被处理但之后将要被处理的大量数据一般存放于高延迟、高容量的内存中。相对于不用分级的内存，用这种分级的内存可以降低延迟，提高计算效率。

CPU 和 GPU 中都有内存分级的设计。相对于 CPU 编程来说，CUDA 编程模型向程序员提供更多的控制权。因此，对 CUDA 编程来说，熟悉其内存的分级组织是非常重要的。

表 6.1 列出了 CUDA 中的几种内存和它们的主要特征。这些特征包括物理位置、设备的访问权限、可见范围及对应变量的生命周期。图 6.1 可以进一步帮助

表 6.1 CUDA 中设备内存的分类与特征

内存类型	物理位置	访问权限	可见范围	生命周期
全局内存	在芯片外	可读可写	所有线程和主机端	由主机分配与释放
常量内存	在芯片外	仅可读	所有线程和主机端	由主机分配与释放
纹理和表面内存	在芯片外	一般仅可读	所有线程和主机端	由主机分配与释放
寄存器内存	在芯片内	可读可写	单个线程	所在线程
局部内存	在芯片外	可读可写	单个线程	所在线程
共享内存	在芯片内	可读可写	单个线程块	所在线程块

理解。本章仅简要介绍 CUDA 中的各种内存，更多细节在后续几章会有进一步讨论。本章会频繁引用前几章讨论过的数组相加的例子。

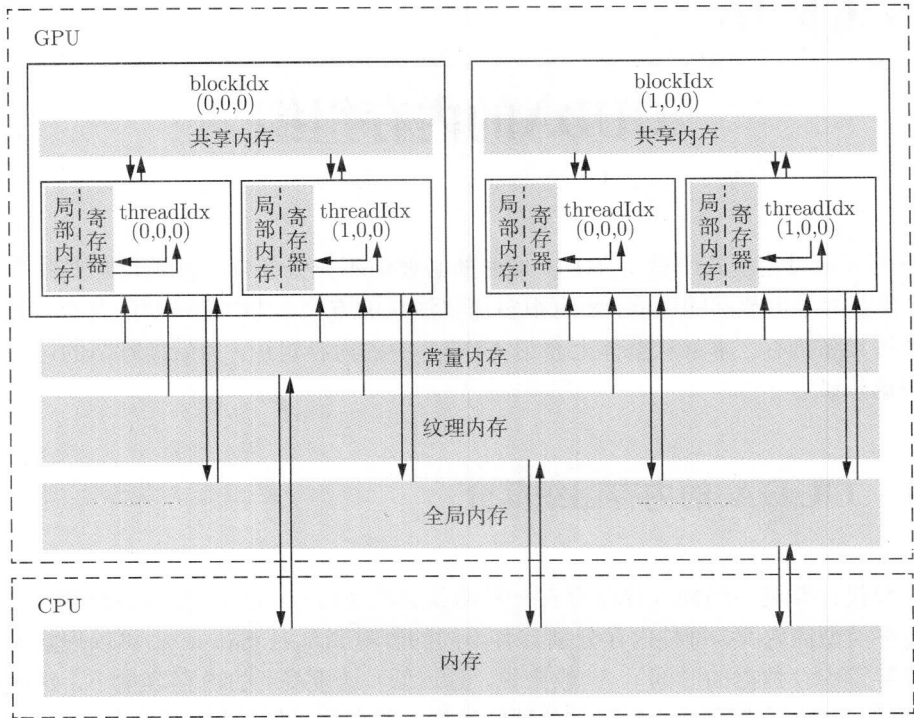


图 6.1 CUDA 中的内存组织示意图
图中箭头的方向表示数据可以移动的方向

6.2 CUDA 中不同类型的内存

6.2.1 全局内存

这里“全局内存”（global memory）的含义是核函数中的所有线程都能够访问其中的数据，和 C++ 中的“全局变量”不是一回事。我们已经用过这种内存，在数组相加的例子中，指针 `d_x`、`d_y` 和 `d_z` 都是指向全局内存的。全局内存由于没有存放在 GPU 的芯片上，因此具有较高的延迟和较低的访问速度。然而，它的容量是所有设备内存中最大的。其容量基本上就是显存容量。表 1.3 列出了几款 GPU 的显存容量。

全局内存的主要角色是为核函数提供数据，并在主机与设备及设备与设备之间传递数据。首先，我们用 `cudaMalloc()` 函数为全局内存变量分配设备内存。然

后，可以直接在核函数中访问分配的内存，改变其中的数据值。我们说过，要尽量减少主机与设备之间的数据传输，但有时是不可避免的。可以用 `cudaMemcpy()` 函数将主机的数据复制到全局内存，或者反过来。在前几章数组相加的例子中，语句

```
cudaMemcpy(d_x, h_x, M, cudaMemcpyHostToDevice);
```

将 M 字节的数据从主机复制到设备，而语句

```
cudaMemcpy(h_z, d_z, M, cudaMemcpyDeviceToHost);
```

将 M 字节的数据从设备复制到主机。还可以将一段全局内存中的数据复制到另一段全局内存中去。例如：

```
cudaMemcpy(d_x, d_y, M, cudaMemcpyDeviceToDevice);
```

的作用就是将首地址为 d_y 的全局内存中 M 字节的数据复制到首地址为 d_x 的全局内存中去。注意：这里必须将数据传输的方向指定为 `cudaMemcpyDeviceToDevice` 或 `cudaMemcpyDefault`。

全局内存可读可写。在数组相加的例子中，语句

```
d_z[n] = d_x[n] + d_y[n];
```

同时体现了全局内存的可读性和可写性。对于线程 n 来说，该语句将变量 d_x 和 d_y 所指全局内存缓冲区的第 n 个元素读出，相加后将结果写入变量 d_z 所指全局内存缓冲区的第 n 个元素。

全局内存对整个网格的所有线程可见。也就是说，一个网格的所有线程都可以访问（读或写）传入核函数的设备指针所指向的全局内存中的全部数据。在上面的语句中，第 n 个线程刚好访问全局内存缓冲区的第 n 个元素，但并不是非要这样。如有需要，第 n 个线程可以访问全局内存缓冲区中的任何一个元素。

全局内存的生命周期（lifetime）不是由核函数决定的，而是由主机端决定的。在数组相加的例子中，由指针 d_x 、 d_y 和 d_z 所指向的全局内存缓冲区的生命周期就是从主机端用 `cudaMalloc()` 对它们分配内存开始，到主机端用 `cudaFree()` 释放它们的内存结束。在这期间，可以在相同的或不同的核函数中多次访问这些全局内存中的数据。

在处理逻辑上的两维或三维问题时，可以用 `cudaMallocPitch()` 和 `cudaMalloc3D()` 函数分配内存，用 `cudaMemcpy2D()` 和 `cudaMemcpy3D()` 复制数据，释放时依然用 `cudaFree()` 函数。本书不讨论这种内存分配函数及相应的数据复制函数。

以上所有的全局内存都称为线性内存（linear memory）。在 CUDA 中还有一种内部构造对用户不透明的（not transparent）全局内存，称为 CUDA Array。CUDA Array 使用英伟达公司不对用户公开的数据排列方式，专为纹理拾取服务，本书不做讨论。

我们前面介绍的全局内存变量都是动态地分配内存的。在 CUDA 中允许使用静态全局内存变量，其所占内存数量是在编译期间就确定的。而且，这样的静态全

局内存变量必须在所有主机与设备函数外部定义, 所以是一种“全局的静态全局内存变量”。这里, 第一个“全局”的含义与 C++ 中全局变量的含义相同, 指的是对应的变量对从其定义之处开始、一个翻译单元内的所有设备函数直接可见。如果采用所谓的分离编译 (separate compiling), 还可以将可见范围进一步扩大, 但本书不讨论分离编译。

静态全局内存变量由以下方式在任何函数外部定义:

```
__device__ T x; // 单个变量
__device__ T y[N]; // 固定长度的数组
```

其中, 修饰符 `__device__` 说明该变量是设备中的变量, 而不是主机中的变量; `T` 是变量的类型; `N` 是一个整型常数。Listing 6.1 展示了静态全局内存变量的使用方式。该程序将输出:

```
d_x = 1, d_y[0] = 11, d_y[1] = 21.
h_y[0] = 11, h_y[1] = 21.
```

在核函数中, 可直接对静态全局内存变量进行访问, 并不需要将它们以参数的形式传给核函数。不可在主机函数中直接访问静态全局内存变量, 但可以用 `cudaMemcpyToSymbol()` 函数和 `cudaMemcpyFromSymbol()` 函数在静态全局内存与主机内存之间传输数据。这两个 CUDA 运行时 API 函数的原型如下:

```
cudaError_t cudaMemcpyToSymbol
(
    const void* symbol, // 静态全局内存变量名
    const void* src, // 主机内存缓冲区指针
    size_t count, // 复制的字节数
    size_t offset = 0, // 从 symbol 对应设备地址开始偏移的字节数
    cudaMemcpyKind kind = cudaMemcpyHostToDevice // 可选参数
);
cudaError_t cudaMemcpyFromSymbol
(
    void* dst, // 主机内存缓冲区指针
    const void* symbol, // 静态全局内存变量名
    size_t count, // 复制的字节数
    size_t offset = 0, // 从 symbol 对应设备地址开始偏移的字节数
    cudaMemcpyKind kind = cudaMemcpyDeviceToHost // 可选参数
);
```

这两个函数的参数 `symbol` 可以是静态全局内存变量的变量名, 也可以是下面要介绍的常量内存变量的变量名。第 16 行调用 `cudaMemcpyToSymbol()` 函数将主

机数组 `h_y` 中的数据复制到静态全局内存数组 `d_y`，第 21 行调用 `cudaMemcpyFromSymbol()` 函数将静态全局内存数组 `d_y` 中的数据复制到主机数组 `h_y`。这里只是展示静态全局内存的使用方法，我们将在第 10 章讨论一种利用静态全局内存加速程序的技巧。

Listing 6.1 本程序 `static.cu` 的全部代码

```

1  #include "error.cuh"
2  #include <stdio.h>
3  __device__ int d_x = 1;
4  __device__ int d_y[2];
5
6  void __global__ my_kernel(void)
7  {
8      d_y[0] += d_x;
9      d_y[1] += d_x;
10     printf("d_x = %d, d_y[0] = %d, d_y[1] = %d.\n", d_x, d_y[0], d_y
        [1]);
11 }
12
13 int main(void)
14 {
15     int h_y[2] = {10, 20};
16     CHECK(cudaMemcpyToSymbol(d_y, h_y, sizeof(int) * 2));
17
18     my_kernel<<<1, 1>>>>();
19     CHECK(cudaDeviceSynchronize());
20
21     CHECK(cudaMemcpyFromSymbol(h_y, d_y, sizeof(int) * 2));
22     printf("h_y[0] = %d, h_y[1] = %d.\n", h_y[0], h_y[1]);
23
24     return 0;
25 }
```

6.2.2 常量内存

常量内存 (constant memory) 是有常量缓存的全局内存，数量有限，仅有 64 KB。它的可见范围和生命周期与全局内存一样。不同的是，常量内存仅可读、不可写。由于有缓存，常量内存的访问速度比全局内存高，但得到高访问速度的前提是一个线程束中的线程（一个线程块中相邻的 32 个线程）要读取相同的常量内存数据。

一个使用常量内存的方法是在核函数外面用 `__constant__` 定义变量，并用前面介绍的 CUDA 运行时 API 函数 `cudaMemcpyToSymbol()` 将数据从主机端复制到设备的常量内存后供核函数使用。当计算能力不小于 2.0 时，给核函数传递的参数（传值，不是像全局变量那样传递指针）就存放在常量内存中，但给核函数传递参数最多只能在一个核函数中使用 4 KB 常量内存。

所以，我们其实已经用过了常量内存。在数组相加的例子中，核函数的参数 `const int N` 就是在主机端定义的变量，并通过传值的方式传送给核函数中的线程使用。在核函数的代码段 `if (n < N)` 中，这个参数 `N` 就被每一个线程使用了。所以，核函数中的每一个线程都知道该变量的值，而且对它的访问比对全局内存的访问要快。除给核函数传递单个的变量外，还可以传递结构体，同样也是使用常量内存。结构体中可以定义单个的变量，也可以定义固定长度的数组。第 13 章的程序将涉及常量内存的使用。

6.2.3 纹理内存和表面内存

纹理内存（texture memory）和表面内存（surface memory）类似于常量内存，也是一种具有缓存的全局内存，有相同的可见范围和生命周期，而且一般仅可读（表面内存也可写）。不同的是，纹理内存和表面内存容量更大，而且使用方式和常量内存也不一样。

对于计算能力不小于 3.5 的 GPU 来说，将某些只读全局内存数据用 `__ldg()` 函数通过只读数据缓存（read-only data cache）读取，既可达到使用纹理内存的加速效果，又可使代码简洁。该函数的原型为

```
T __ldg(const T* address);
```

其中，`T` 是需要读取的数据的类型；`address` 是数据的地址。对帕斯卡架构和更高的架构来说，全局内存的读取在默认情况下就利用了 `__ldg()` 函数，所以不需要明显地使用它。我们在第 7 章就会讨论该函数的使用。

6.2.4 寄存器

在核函数中定义的不加任何限定符的变量一般来说就存放于寄存器（register）中。核函数中定义的不加任何限定符的数组有可能存放于寄存器中，但也有可能存放于局部内存中。另外，以前提到过的各种内建变量，如 `gridDim`、`blockDim`、`blockIdx`、`threadIdx` 及 `warpSize` 都保存在特殊的寄存器中。在核函数中访问这些内建变量是很高效的。

我们已经使用过寄存器变量。在数组求和的例子中，我们在核函数中有如下语句：


```
const int n = blockDim.x * blockIdx.x + threadIdx.x;
```

这里的 `n` 就是一个寄存器变量。寄存器可读可写。上述语句的作用就是定义一个寄存器变量 `n` 并将赋值号右边计算出来的值赋给它（写入）。在稍后的语句

```
z[n] = x[n] + y[n];
```

中，寄存器变量 `n` 的值被使用（读出）。

寄存器变量仅仅被一个线程可见。也就是说，每一个线程都有一个变量 `n` 的副本。虽然在核函数的代码中用了同一个变量名，但是不同的线程中该寄存器变量的值是可以不同的。每个线程都只能对它的副本进行读写。寄存器的生命周期也与所属线程的生命周期一致，从定义它开始，到线程消失时结束。

寄存器内存在芯片上（on-chip），是所有内存中访问速度最高的，但是其数量也很有限。表 6.2 列出了几个不同计算能力的 GPU 中与寄存器和后面要介绍的共享内存有关的技术指标。该表只包含少数几个计算能力，更完整的列表见《CUDA C++ Programming Guide》的附录 H。一个寄存器占有 32 bit（4 字节）的内存。所以，一个双精度浮点数将使用两个寄存器。这是在估算寄存器使用量时要注意的。

6.2.5 局部内存

我们还没有用过局部内存（local memory），但从用法上看，局部内存和寄存器几乎一样。核函数中定义的不加任何限定符的变量有可能在寄存器中，也有可能局部内存中。寄存器中放不下的变量，以及索引值不能在编译时就确定的数组，都有可能放在局部内存中。这种判断是由编译器自动做的。对于数组相加例子中的变量 `n` 来说，作者可以肯定它在寄存器中，而不是局部内存中，因为核函数所用寄存器数量还远远没有达到上限。

虽然局部内存存在用法上类似于寄存器，但从硬件来看，局部内存只是全局内存的一部分。所以，局部内存的延迟也很高。每个线程最多能使用高达 512 KB 的局部内存，但使用过多会降低程序的性能。

6.2.6 共享内存

我们还没有使用过共享内存（shared memory）。共享内存和寄存器类似，存在于芯片上，具有仅次于寄存器的读写速度，数量也有限。表 6.2 列出了与几个计算能力对应的共享内存数量指标。

不同于寄存器的是，共享内存对整个线程块可见，其生命周期也与整个线程块一致。也就是说，每个线程块拥有一个共享内存变量的副本。共享内存变量的值在不同的线程块中可以不同。一个线程块中的所有线程都可以访问该线程块的共享内存变量副本，但是不能访问其他线程块的共享内存变量副本。共享内存的主要作

用是减少对全局内存的访问，或者改善对全局内存的访问模式。这些将在第 8 章详细地讨论。

表 6.2 几个计算能力的技术指标

计算能力	3.5	6.0	7.0	7.5
GPU 代表	Tesla K40	Tesla P100	Tesla V100	Geforce RTX 2080
SM 寄存器数上限	64 K	64 K	64 K	64 K
单个线程块寄存器数上限	64 K	64 K	64 K	64 K
单个线程寄存器数上限	255	255	255	255
SM 共享内存上限/KB	48	64	96	64
单个线程块共享内存上限/KB	48	48	96	64

6.2.7 L1 和 L2 缓存

从费米架构开始，有了 SM 层次的 L1 缓存（一级缓存）和设备（一个设备有多个 SM）层次的 L2 缓存（二级缓存）。它们主要用来缓存全局内存和局部内存的访问，减少延迟。

从硬件的角度来看，开普勒架构中的 L1 缓存和共享内存使用同一块物理芯片；在麦克斯韦架构和帕斯卡架构中，L1 缓存和纹理缓存统一起来，而共享内存是独立的；在伏特架构和图灵架构中，L1 缓存、纹理缓存及共享内存三者统一起来。从编程的角度来看，共享内存是可编程的缓存（共享内存的使用完全由用户操控），而 L1 和 L2 缓存是不可编程的缓存（用户最多能引导编译器做一些选择）。

对某些架构来说，还可以针对单个核函数或者整个程序改变 L1 缓存和共享内存的比例。具体地说：

(1) 计算能力 3.5：L1 缓存和共享内存共有 64 KB，可以将共享内存上限设置成 16 KB、32 KB 和 48 KB，其余的归 L1 缓存。默认情况下有 48KB 共享内存。

(2) 计算能力 3.7：L1 缓存和共享内存共有 128 KB，可以将共享内存上限设置成 80 KB、96 KB 和 112 KB，其余的归 L1 缓存。默认情况下有 112 KB 共享内存。

(3) 麦克斯韦架构和帕斯卡架构不允许调整共享内存的上限。

(4) 伏特架构：统一的（L1/纹理/共享内存）缓存共有 128 KB，共享内存上限可调整为 0 KB、8 KB、16 KB、32 KB、64 KB 或 96 KB。

(5) 图灵架构：统一的（L1/纹理/共享内存）缓存共有 96 KB，共享内存上限可调整为 32 KB 或 64 KB。

由于以上关于共享内存比例的设置不是很通用，本书不对它们做进一步讨论。感兴趣的读者可阅读《CUDA C++ Programming Guide》和其他资料进一步学习。

6.3 SM 及其占有率

6.3.1 SM 的构成

我们在第 5 章讨论并行规模对 CUDA 程序性能的影响时提到了流多处理器 SM。一个 GPU 是由多个 SM 构成的。一个 SM 包含如下资源：

- (1) 一定数量的寄存器（参见表 6.2）。
- (2) 一定数量的共享内存（参见表 6.2）。
- (3) 常量内存的缓存。
- (4) 纹理和表面内存的缓存。
- (5) L1 缓存。
- (6) 两个（计算能力 6.0）或 4 个（其他计算能力）线程束调度器（warp scheduler），用于在不同线程的上下文之间迅速地切换，以及为准备就绪的线程束发出执行指令。
- (7) 执行核心，包括：
 - 1) 若干整型数运算的核心（INT32）。
 - 2) 若干单精度浮点数运算的核心（FP32）。
 - 3) 若干双精度浮点数运算的核心（FP64）。
 - 4) 若干单精度浮点数超越函数（transcendental functions）的特殊函数单元（special function units, SFUs）。
 - 5) 若干混合精度的张量核心（tensor cores，由伏特架构引入，适用于机器学习中的低精度矩阵计算，本书不讨论）。

6.3.2 SM 的占有率

因为一个 SM 中的各种计算资源是有限的，那么有些情况下一个 SM 中驻留的线程数目就有可能达不到理想的最大值。此时，我们说该 SM 的占有率小于 100%。获得 100% 的占有率并不是获得高性能的必要或充分条件，但一般来说，要尽量让 SM 的占有率不小于某个值，如 25%，才有可能获得较高的性能。

在第 5 章，我们讨论了并行规模。当并行规模较小时，有些 SM 可能就没有被利用，占有率为零。这是导致程序性能低下的原因之一。当并行规模足够大时，也有可能得到非 100% 的占有率，这就是下面要讨论的情形。

在表 6.2 中，我们列举了一个 SM、一个线程块及一个线程中能够使用的寄存器和共享内存的上限。在第 2 章，我们还提到了，一个线程块（无论几维的）中的线程数不能超过 1024。要分析 SM 的理论占有率（theoretical occupancy），还需要

知道两个指标:

(1) 一个 SM 中最多能拥有的线程块个数为 $N_b = 16$ (开普勒架构和图灵架构) 或者 $N_b = 32$ (麦克斯韦架构、帕斯卡架构和伏特架构)。

(2) 一个 SM 中最多能拥有的线程个数为 $N_t = 2048$ (从开普勒架构到伏特架构) 或者 $N_t = 1024$ (图灵架构)。

下面, 在并行规模足够大 (即核函数执行配置中定义的总线程数足够多) 的前提下分几种情况来分析 SM 的理论占有率:

(1) 寄存器和共享内存使用量很少的情况。此时, SM 的占有率完全由执行配置中的线程块大小决定。关于线程块大小, 读者也许注意到我们之前总是用 128。这是因为, SM 中线程的执行是以线程束为单位的, 所以最好将线程块大小取为线程束大小 (32 个线程) 的整数倍。例如, 假设将线程块大小定义为 100, 那么一个线程块中将有 3 个完整的线程束 (一共 96 个线程) 和一个不完整的线程束 (只有 4 个线程)。在执行核函数中的指令时, 不完整的线程束花的时间和完整的线程束花的时间一样, 这就无形中浪费了计算资源。所以, 建议将线程块大小取为 32 的整数倍。在该前提下, 任何不小于 N_t/N_b 而且能整除 N_t 的线程块大小都能得到 100% 的占有率。根据我们列出的数据, 线程块大小不小于 128 时开普勒架构能获得 100% 的占有率; 线程块大小不小于 64 时其他架构能获得 100% 的占有率。作者近几年都用一块开普勒架构的 Tesla K40 开发程序, 所以习惯了在一般情况下用 128 的线程块大小。

(2) 有限的寄存器数量对占有率的约束情况。我们只针对表 6.2 中列出的几个计算能力进行分析, 读者可类似地分析其他未列出的计算能力。对于表 6.2 中列出的所有计算能力, 一个 SM 最多能使用的寄存器个数为 64 K (64×1024)。除图灵架构外, 如果我们希望在一个 SM 中驻留最多的线程 (2048 个), 核函数中的每个线程最多只能用 32 个寄存器。当每个线程所用寄存器个数大于 64 时, SM 的占有率将小于 50%; 当每个线程所用寄存器个数大于 128 时, SM 的占有率将小于 25%。对于图灵架构, 同样的占有率允许使用更多的寄存器。

(3) 有限的共享内存对占有率的约束情况。因为共享内存的数量随着计算能力的上升没有显著的变化规律, 所以我们这里仅针对计算能力 3.5 进行分析, 对其他计算能力可以类似地分析。如果线程块大小为 128, 那么每个 SM 要激活 16 个线程块才能有 2048 个线程, 达到 100% 的占有率。此时, 一个线程块最多能使用 3 KB 的共享内存。在不改变线程块大小的情况下, 要达到 50% 的占有率, 一个线程块最多能使用 6 KB 的共享内存; 要达到 25% 的占有率, 一个线程块最多能使用 12 KB 的共享内存。如果一个线程块使用了超过 48 KB 的共享内存, 会直接导致核函数无法运行。对其他线程块大小可类似地分析。

以上单独分析了线程块大小、寄存器数量及共享内存数量对 SM 占有率的影响。一般情况下，需要综合以上 3 点分析。在 CUDA 工具箱中，有一个名为 CUDA_Occupancy_Calculator.xls 的 Excel 文档，可用来计算各种情况下的 SM 占有率，感兴趣的读者可以去尝试使用。

值得一提的是，用编译器选项 `--ptxas-options=-v` 可以报道每个核函数的寄存器使用数量。CUDA 还提供了核函数的 `__launch_bounds__()` 修饰符和 `--maxrregcount=` 编译选项来让用户分别对一个核函数和所有核函数中寄存器的使用数量进行控制。本书不对此展开讨论，感兴趣的读者可查阅其他资料进一步学习。

6.4 用 CUDA 运行时 API 函数查询设备

在第 1 章，我们介绍了如何利用 `nvidia-smi` 程序对设备进行某些方面的查询与设置。本节介绍用 CUDA 运行时 API 函数查询所用 GPU 的规格。Listing 6.2 所示程序可用来查询一些前面介绍过的 GPU 规格。

Listing 6.2 本程序 query.cu 的全部代码

```

1  #include "error.cuh"
2  #include <stdio.h>
3
4  int main(int argc, char *argv[])
5  {
6      int device_id = 0;
7      if (argc > 1) device_id = atoi(argv[1]);
8      CHECK(cudaSetDevice(device_id));
9
10     cudaDeviceProp prop;
11     CHECK(cudaGetDeviceProperties(&prop, device_id));
12
13     printf("Device id:                                %d\n",
14            device_id);
15     printf("Device name:                                %s\n",
16            prop.name);
17     printf("Compute capability:                                %d.%d\n",
18            prop.major, prop.minor);
19     printf("Amount of global memory:                            %g GB\n",
20            prop.totalGlobalMem / (1024.0 * 1024 * 1024));
21     printf("Amount of constant memory:                            %g KB\n",
22            prop.totalConstMem / 1024.0);

```

```

23     printf("Maximum grid size:                %d %d %d\n",
24            prop.maxGridSize[0],
25            prop.maxGridSize[1], prop.maxGridSize[2]);
26     printf("Maximum block size:                %d %d %d\n",
27            prop.maxThreadsDim[0], prop.maxThreadsDim[1],
28            prop.maxThreadsDim[2]);
29     printf("Number of SMs:                    %d\n",
30            prop.multiProcessorCount);
31     printf("Maximum amount of shared memory per block: %g KB\n",
32            prop.sharedMemPerBlock / 1024.0);
33     printf("Maximum amount of shared memory per SM:   %g KB\n",
34            prop.sharedMemPerMultiprocessor / 1024.0);
35     printf("Maximum number of registers per block:    %d K\n",
36            prop.regsPerBlock / 1024);
37     printf("Maximum number of registers per SM:       %d K\n",
38            prop.regsPerMultiprocessor / 1024);
39     printf("Maximum number of threads per block:      %d\n",
40            prop.maxThreadsPerBlock);
41     printf("Maximum number of threads per SM:         %d\n",
42            prop.maxThreadsPerMultiProcessor);
43
44     return 0;
45 }

```

程序的第 10 行定义了一个 CUDA 中定义好的结构体类型 `cudaDeviceProp` 的变量 `prop`。在第 11 行，利用 CUDA 运行时 API 函数 `cudaGetDeviceProperties()` 得到了编号为 `device_id` 的设备的性质，存放在结构体变量 `prop` 中。从第 13 行开始，将变量 `prop` 中的某些成员的值输出。在装有 GeForce RTX 2070 的计算机中得到如下输出：

```

Device id:                0
Device name:              GeForce RTX 2070 with
                           Max-Q Design
Compute capability:       7.5
Amount of global memory:  8 GB
Amount of constant memory: 64 KB
Maximum grid size:       2147483647 65535 65535
Maximum block size:      1024 1024 64
Number of SMs:           36
Maximum amount of shared memory per block: 48 KB

```

Maximum amount of shared memory per SM:	64 KB
Maximum number of registers per block:	64 K
Maximum number of registers per SM:	64 K
Maximum number of threads per block:	1024
Maximum number of threads per SM:	1024

读者可以尝试在自己的系统中运行该程序, 确保能够理解每一个规格的含义。在本例中, 我们默认选择查询编号为 0 的设备。如果读者的系统中有不止一块 GPU, 而且不想查询第 0 号设备, 则可在运行程序时通过命令行参数指定一个设备编号。第 8 行的 `cudaSetDevice()` 函数将对所指定的设备进行初始化。

另外, 读者还可以回顾一下在第 1 章介绍过的用 `nvidia-smi` 程序在命令行选择 GPU 的方法。在 CUDA 工具箱中, 有一个名为 `deviceQuery.cu` 的程序, 可以输出更多的信息。

第 12 章

使用统一内存编程

在之前的章节中，我们学会了如何分配与释放显存、在主机与设备之间传输数据及调用核函数进行计算。本章将学习一种新编程模型：统一内存（unified memory）编程。在这种新的编程模型下，将不再需要手动地在主机与设备间传输数据。统一内存编程模型由 CUDA 6 引入，从开普勒架构开始就可用，而且其易用程度和性能都随着 GPU 架构的更新有所提高。然而，到目前为止，统一内存的某些功能在 Windows 操作系统中依然受到限制，本章的部分例子也只能在 Linux 系统中通过测试。

在开始本章的学习之前，我们强调一点：本章的内容对使用 CUDA 进行程序开发来说并不是必需的。一个 CUDA 项目完全可以不使用统一内存。所以，在第一次阅读本书时，可以选择性地跳过本章。本书后面的章节基本上不会涉及统一内存。

12.1 统一内存简介

12.1.1 统一内存的基本概念

统一内存是一种逻辑上的概念，它既不是显存，又不是主机的内存，而是一种系统中的任何处理器（CPU 或者 GPU）都可以访问，并能保证一致性的虚拟存储器。这种虚拟存储器是通过 CPU 和 GPU 各自内部集成的内存管理单元（memory management unit）实现的。在某种程度上，可以将一个系统中某个处理器的内存看成整个统一内存的超级大缓存。

统一内存编程从开普勒架构起就受到支持，但开普勒架构和麦克斯韦架构的 GPU 提供的统一内存编程功能相对较弱。从帕斯卡架构到现在的伏特架构和图灵架构，统一内存的功能加强了很多，主要是因为这些架构的 GPU 具有了精细的缺页异常处理（page-fault handling）能力。一般把开普勒架构和麦克斯韦架构的统一内存称为第一代统一内存，把从帕斯卡架构开始的统一内存称为第二代统一内存。

在统一内存之前，还有一种零复制内存（zero-copy memory）。它们都提供了一种统一的能被 CPU 和 GPU 都访问到的存储器，但零复制内存只是用主机内存作

为存储介质，而统一内存则能将数据放在一个最合适的地方（可以是主机，也可以是设备）。这里我们不介绍零复制内存的使用方法。

另外，统一内存编程在很大程度上涉及多 GPU 编程。由于本书不涉及多 GPU 编程，故也不对这方面的内容进行介绍。

12.1.2 使用统一内存对硬件的要求

使用统一内存对硬件有较高的要求：

(1) 对于所有的功能，GPU 的架构都必须不低于开普勒架构，主机应用程序必须为 64 位的。

(2) 对于一些较新的功能（在适当的时候会具体指出），至少需要帕斯卡架构的 GPU，而且主机要用 Linux 系统。也就是说，无论用什么 GPU，在 Windows 系统中都只能用第一代统一内存的功能。

(3) 在具有 IBM Power 9 和 NVLink 的系统中，伏特架构的 GPU 支持设备访问任何主机内存，包括用 malloc 分配的动态数组和在栈上分配的静态数组。这是非常诱人的特征，但普通用户（包括作者）可能没有这样的硬件资源。

12.1.3 统一内存编程的优势

下面是使用统一内存可能带来的好处：

(1) 统一内存使 CUDA 编程更加简单。使用统一内存，将不再需要手动将数据在主机与设备之间传输，也不需要针对同一组数据定义两个指针，并分别分配主机和设备内存。对于某个统一内存变量，可以直接从 GPU 或者 CPU 中进行访问。

(2) 可能会提供比手工移动数据更好的性能。底层的统一内存实现，可能会自动将一部分数据放置到离某个存储器更近的位置（如部分放置到某卡的显存中，部分放置到内存中），这种自动的就近数据存放，有可能提升性能。

(3) 允许 GPU 在使用了统一内存的情况下，进行超量分配。超出 GPU 内存额度的部分可能存放在主机上。这可能是使用统一内存时最大的好处，因为一般来说 CPU 的内存可以更多，但处理速度较低，而 GPU 虽然处理速度较高，但内存（显存）数量有限（参看第 1 章中所列的数据）。该功能要求帕斯卡架构或更高的架构及 Linux 操作系统。

12.2 统一内存的基本使用方法

统一内存存在设备中是当作全局内存使用的，而且必须在主机端定义或分配内存，而不能在设备端（核函数和 `__device__()` 函数）定义或分配内存。例如，在核

函数中由 malloc 分配的堆内存不属于统一内存，从而无法被 CPU 访问。下面介绍统一内存的基本用法。

12.2.1 动态统一内存

我们继续研究由第 3 章引入的数组相加的例子。用统一内存改写后，主函数中的内容见 Listing 12.1。我们这里省略核函数和一个检查结果正确性的主机端函数的定义，因为它们相对于之前未使用统一内存的情况没有任何变化。

Listing 12.1 本程序 add.cu 中的主函数

```

1  int main(void)
2  {
3      const int N = 100000000;
4      const int M = sizeof(double) * N;
5      double *x, *y, *z;
6      CHECK(cudaMallocManaged((void **)&x, M));
7      CHECK(cudaMallocManaged((void **)&y, M));
8      CHECK(cudaMallocManaged((void **)&z, M));
9
10     for (int n = 0; n < N; ++n)
11     {
12         x[n] = a;
13         y[n] = b;
14     }
15
16     const int block_size = 128;
17     const int grid_size = N / block_size;
18     add<<<grid_size, block_size>>>(x, y, z);
19
20     CHECK(cudaDeviceSynchronize());
21     check(z, N);
22
23     CHECK(cudaFree(x));
24     CHECK(cudaFree(y));
25     CHECK(cudaFree(z));
26     return 0;
27 }
```

程序的第 5 行定义了 3 个双精度浮点数类型变量的指针，第 6~8 行给它们分配了统一内存。这里使用了一个新的 CUDA 运行时 API 函数，原型为
 cudaError_t cudaMallocManaged

```
(
    void **devPtr,
    size_t size,
    unsigned flags = 0
);
```

相比 `cudaMalloc()` 函数，该函数多了一个可选参数 `flags`。该参数的默认值是 `cudaMemAttachGlobal`。如果取默认值，代表分配的全局内存可由任何设备通过任何 CUDA 流访问。`flags` 另一个可取的值是 `cudaMemAttachHost`，但我们不讨论这种情形。统一内存的释放依然是用之前用过的 `cudaFree()` 函数。值得强调的是，只能在主机端使用该函数分配统一内存，而不能在核函数中使用该函数。

分配了统一内存的变量既可以被设备访问，又可以被主机访问。第 10~14 行，在主机端对统一内存变量赋初值。第 18 行，调用核函数在设备端对统一内存进行访问。从核函数的角度来看，统一内存和普通设备内存使用上没有任何区别。也就是说，在将一个程序从不使用统一内存的版本改为使用统一内存的版本（或者反过来）时，不需要对核函数进行修改。而且，可以一点一点地将非统一内存改为统一内存（或者反过来），即同一个程序中可以同时使用统一内存和非统一内存。

对于开普勒架构和麦克斯韦架构（第一代统一内存）来说，主机与设备不能并发地访问统一内存。又因为核函数的调用是异步的，故在调用任何核函数之后，不能紧接着从 CPU 访问任何统一内存变量（不管核函数是否访问了 CPU 将要访问的某段统一内存地址），必须在中间加上一个同步函数，以确保核函数对统一内存的访问已经结束，如第 20 行的 `cudaDeviceSynchronize()`。在主机与设备之间对非统一内存的数据传输也可起到同步设备与主机的效果。对于帕斯卡架构及更高的架构（第二代统一内存），使用 Linux 系统时则不需要这样的同步操作。这是第一代统一内存和第二代统一内存的重大区别之一。再强调一次，目前无论用什么 GPU，在 Windows 系统中都只能用第一代统一内存的功能。

比较这个例子和第 3 章的 `add1.cu` 程序，能看出使用了统一内存后，程序确实简化了许多，不需要再针对同一组数据定义两个数组（一个在主机，一个在设备），而且不需要显式地进行主机与设备间的数据传输。至于程序的性能，很难通过这种简单的程序进行测试。如果只是针对核函数来说，可以说两个版本的核函数具有同样的性能。

12.2.2 静态统一内存

正如 GPU 中的全局内存除可以动态分配外，还可以静态分配，统一内存也可以静态地分配。要定义静态统一内存，只需要在修饰符 `__device__` 的基础上再

加上修饰符 `__managed__` 即可。注意：这样的变量是在任何函数外部定义的，可见范围是所在源文件（更准确地说是所在翻译单元）。下面的 Listing 12.2 来自于《CUDA C++ Programming Guide》。

Listing 12.2 本章静态统一内存的程序示例

```

1  #include <stdio.h>
2
3  __device__ __managed__ int ret[1000];
4
5  __global__ void AplusB(int a, int b)
6  {
7      ret[threadIdx.x] = a + b + threadIdx.x;
8  }
9
10 int main()
11 {
12     AplusB<<<1, 1000>>>(10, 100);
13     cudaDeviceSynchronize(); //第一代统一内存或者 Windows 系统需要
14     for(int i = 0; i < 1000; i++)
15     {
16         printf("%d: A+B = %d\n", i, ret[i]);
17     }
18     return 0;
19 }
```

12.3 使用统一内存申请超量的内存

使用统一内存的一个好处是在适当的时候可以超量申请设备内存。我们用几个程序对此进行测试。该功能要求 GPU 的架构不低于帕斯卡架构（即属于第二代统一内存的功能）。这里用一台带有图灵架构的 GeForce RTX 2070 的计算机来测试。注意：该功能目前无法在 Windows 操作系统中使用。

12.3.1 第一个测试

我们的第一个测试代码见 Listing 12.3。如果用如下方式编译：

```
nvcc -arch=sm_75 -O3 oversubscription1.cu
```

得到的可执行文件是不使用统一内存的版本。运行得到的可执行文件，作者得到的输出为

Allocate 1 GB device memory.
 Allocate 2 GB device memory.
 Allocate 3 GB device memory.
 Allocate 4 GB device memory.
 Allocate 5 GB device memory.
 Allocate 6 GB device memory.
 Allocate 7 GB device memory.

CUDA Error:

File: oversubscription1.cu
 Line: 18
 Error code: 2
 Error text: out of memory

Listing 12.3 本章程序 oversubscription1.cu 的内容

```

1  #include "error.cuh"
2  #include <stdio.h>
3  #include <stdint.h>
4
5  const int N = 30;
6
7  int main(void)
8  {
9      for (int n = 1; n <= N; ++n)
10     {
11         const size_t size = size_t(n) * 1024 * 1024 * 1024;
12         uint64_t *x;
13     #ifdef UNIFIED
14         CHECK(cudaMallocManaged(&x, size));
15         CHECK(cudaFree(x));
16         printf("Allocated %d GB unified memory without touch.\n", n);
17     #else
18         CHECK(cudaMalloc(&x, size));
19         CHECK(cudaFree(x));
20         printf("Allocate %d GB device memory.\n", n);
21     #endif
22     }
23     return 0;
24 }
```

因为 GeForce RTX 2070 只有不到 8 GB 的设备内存，所以程序能够用函数

`cudaMalloc()` 分配 7 GB 的设备内存，但在尝试分配 8 GB 的设备内存时失败。如果用如下方式编译：

```
nvcc -arch=sm_75 -O3 -DUNIFIED oversubscription1.cu
```

得到的可执行文件是使用统一内存的版本。运行得到的可执行文件，程序不会报错，而且显示可以分配 30 GB 的统一内存：

```
Allocated 1 GB unified memory without touch.
```

```
Allocated 2 GB unified memory without touch.
```

```
...
```

```
Allocated 30 GB unified memory without touch.
```

作者的计算机只有 16 GB 的主机内存，加上显存一共不到 24 GB。为什么该测试程序显示可以分配 30 GB 的统一内存呢？实际上，对于第二代统一内存来说，函数 `cudaMallocManaged()` 的成功调用只代表成功地预定了一段地址空间，而统一内存的实际分配发生在主机或者设备第一次访问预留的内存时。在上述测试例子中，我们在调用 `cudaMallocManaged()` 分配统一内存之后，在调用 `cudaFree()` 释放统一内存之前，没有做任何事情，实际上相当于没有真正地分配统一内存。下面，我们分别测试在这两个函数之间用主机和设备访问统一内存的情形。

12.3.2 第二个测试

我们在上述程序的基础上稍做修改，得到如 Listing 12.4 所示的程序。在主函数中，我们在调用 `cudaMallocManaged()` 分配统一内存之后，在调用 `cudaFree()` 释放统一内存之前，调用了自定义的 `gpu_touch()` 核函数让设备对统一内存中的数据进行初始化。编译后在作者的计算机中运行该程序，在显示能成功分配 20 GB 的统一内存之后报道如下错误信息：

```
CUDA Error:
```

```
File:      oversubscription2.cu
```

```
Line:      25
```

```
Error code: 77
```

```
Error text: an illegal memory access was encountered
```

该测试说明，在 GPU 中对统一内存进行初始化时触发了统一内存的真正分配。虽然我们的主机与设备内存之和约为 24 GB，但由于操作系统和其他应用程序需要使用一定数量的内存，程序在尝试分配 21 GB 的统一内存时失败。

Listing 12.4 本章程序 `oversubscription2.cu` 的内容

```
1 #include "error.cuh"
2 #include <stdio.h>
```

```

3  #include <stdint.h>
4
5  const int N = 30;
6
7  __global__ void gpu_touch(uint64_t *x, const size_t size)
8  {
9      const size_t i = blockIdx.x * blockDim.x + threadIdx.x;
10     if (i < size)
11     {
12         x[i] = 0;
13     }
14 }
15
16 int main(void)
17 {
18     for (int n = 1; n <= N; ++n)
19     {
20         const size_t size = size_t(n) * 1024 * 1024 * 1024;
21         uint64_t *x;
22         CHECK(cudaMallocManaged(&x, size));
23         gpu_touch<<<size / sizeof(uint64_t) / 1024, 1024>>>(x, size);
24         CHECK(cudaGetLastError());
25         CHECK(cudaDeviceSynchronize());
26         CHECK(cudaFree(x));
27         printf("Allocated %d GB unified memory with GPU touch.\n", n);
28     }
29     return 0;
30 }

```

12.3.3 第三个测试

因为主机也可以访问统一内存，我们也可以尝试在主机端初始化统一内存中的数据。在前一测试程序的基础上，我们写出如 Listing 12.5 所示的程序。在主函数中，我们在调用 `cudaMallocManaged()` 分配统一内存之后，在调用 `cudaFree()` 释放统一内存之前，调用了自定义的 `cpu_touch()` 函数让主机对统一内存中的数据进行初始化。编译后在作者的计算机中运行该程序，结果显示最多能成功分配 13 GB 的统一内存，之后抛出一个错误信息“Killed”。这说明，仅仅在 CPU 中访问统一内存的话，在使用完主机内存后不会自动使用设备内存。所以，若要将主机和设备的内存都纳入统一内存，则需要及时地让 GPU 访问统一内存或者用 12.4 节介绍的 `cudaMemPrefetchAsync()` 函数实现数据从主机到设备的迁移。

Listing 12.5 本程序 oversubscription3.cu 的内容

```

1  #include "error.cuh"
2  #include <stdio.h>
3  #include <stdint.h>
4
5  const int N = 30;
6
7  void cpu_touch(uint64_t *x, size_t size)
8  {
9      for (size_t i = 0; i < size / sizeof(uint64_t); i++)
10     {
11         x[i] = 0;
12     }
13 }
14
15 int main(void)
16 {
17     for (int n = 1; n <= N; ++n)
18     {
19         size_t size = size_t(n) * 1024 * 1024 * 1024;
20         uint64_t *x;
21         CHECK(cudaMallocManaged(&x, size));
22         cpu_touch(x, size);
23         CHECK(cudaFree(x));
24         printf("Allocated %d GB unified memory with CPU touch.\n", n);
25     }
26     return 0;
27 }

```

12.4 优化使用统一内存的程序

为了在使用统一内存时获得较高性能，需要避免缺页异常、保持数据的局部性（让相关数据尽量靠近对应的处理器）但避免内存抖动（即频繁地在不同的处理器之间传输数据）。CUDA 的统一内存机制可以部分地自动做到这些，但很多情况下还是需要手动地给编译器一些提示（hints），如使用 CUDA 运行时 API 函数 `cudaMemAdvise()` 和 `cudaMemPrefetchAsync()`。我们这里仅介绍后者，并用一个例子展示其用法。该部分的功能由第二代统一内存引入，故需要使用 Linux 操作系统和不低于帕斯卡架构的 GPU。

函数 `cudaMemPrefetchAsync()` 的原型如下:

```
cudaError_t cudaMemPrefetchAsync
(
    const void *devPtr,
    size_t count,
    int dstDevice,
    cudaStream_t stream
);
```

该函数的作用是在 CUDA 流 `stream` 中将统一内存缓冲区 `devPtr` 内 `count` 字节的内存迁移到设备 `dstDevice` (主机的设备号用 `cudaCpuDeviceId` 表示) 中的内存区域, 从而防止 (或减少) 缺页异常, 并提高数据的局部性。

Listing 12.6 展示了如何在本章的数组相加程序中加入函数 `cudaMemPrefetchAsync()` 的调用。在第 3~4 行, 我们用 CUDA 运行时 API 函数 `cudaGetDevice()` 获得了当前活跃的主机线程所使用的设备号。注意: 这里给函数 `cudaGetDevice()` 传递的是一个整型变量的指针。在执行核函数之前, 第 22~24 行调用 `cudaMemPrefetchAsync()` 函数, 实现统一内存从主机到设备的迁移。在调用 `check()` 函数从主机访问统一内存之前, 第 28 行继续调用 `cudaMemPrefetchAsync()` 函数, 实现统一内存从设备到主机的迁移。该程序没有使用非默认流, 故 `cudaMemPrefetchAsync()` 函数中的最后一个参数为默认流 `NULL`。最后值得注意的是, 我们去掉了紧跟核函数的 `cudaDeviceSynchronize()` 函数调用, 因为在使用第二代统一内存时不需要该同步操作。

Listing 12.6 本程序 `prefetch.cu` 中的主函数

```
1  int main(void)
2  {
3      int device_id = 0;
4      CHECK(cudaGetDevice(&device_id));
5
6      const int N = 100000000;
7      const int M = sizeof(double) * N;
8      double *x, *y, *z;
9      CHECK(cudaMallocManaged((void **)&x, M));
10     CHECK(cudaMallocManaged((void **)&y, M));
11     CHECK(cudaMallocManaged((void **)&z, M));
12
13     for (int n = 0; n < N; ++n)
14     {
15         x[n] = a;
```

```
16     y[n] = b;
17 }
18
19     const int block_size = 128;
20     const int grid_size = N / block_size;
21
22     CHECK(cudaMemPrefetchAsync(x, M, device_id, NULL));
23     CHECK(cudaMemPrefetchAsync(y, M, device_id, NULL));
24     CHECK(cudaMemPrefetchAsync(z, M, device_id, NULL));
25
26     add<<<grid_size, block_size>>>(x, y, z);
27
28     CHECK(cudaMemPrefetchAsync(z, M, cudaCpuDeviceId, NULL));
29
30     check(z, N);
31
32     CHECK(cudaFree(x));
33     CHECK(cudaFree(y));
34     CHECK(cudaFree(z));
35     return 0;
36 }
```

一般来说，在使用统一内存时，要尽可能多地使用 `cudaMemPrefetchAsync()` 函数，将缺页异常的次数最小化。即使需要在程序中插入这种对编译器的提示语句，使用统一内存的程序从整体上来说还是比不使用统一内存的程序要简洁。限于篇幅，本书不再进一步讨论统一内存的用法，请感兴趣的读者参阅《CUDA C++ Programming Guide》的附录 K。至于是否应该尽可能多地使用统一内存，不同的人可能有不同的看法。我们留给读者自己在实践中去体会其中的优缺点。